Praktikum 1 – Isim Simulator

NRP	: 1. 3223600038
	2. 3223600041
	3. 3223600048
	4. 3223600050
	5. 3223600058
Nama	: 1. Karolus Wira Wicaksana
	2. Akmal Rafiuddin
	3. Moch. Faris Afif
	4. Labib Al Afaf
	5. Ihsanta Zaki Sanjaya
Kelas	: 2 Teknik Komputer B
Matakuliah	: Praktikum Desain dan Organisasi Komputer
Tanggal	: Rabu, 19 Februari 2025
Dosen	: Dr. Setiawardhana, ST. MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Isim Simulator

A. Tujuan

a. Mahasiswa dapat membuat program untuk rangkaian kombinasional dan rangkaian sekuensial menggunakan software ISE xilinx.
b. Mahasiswa dapat menampilkan RTL schematic dari rangkaian yang dibuat.
c. Mahasiswa dapat mensimulasikan menggunakaan Isim simulator.

B. Dasar Teori

Ketika sebuah rancangan rangkaian diimplementasikan ke dalam FPGA, baik menggunakan metode schematic atau metode HDL, hendaknya perlu diuji agar kita dapat mengetahui hasil/output dari rangkaian tersebut. Proses pengujian rancangan rangkaian ini sering disebut proses simulasi. Melalui proses simulasi, dapat mengetahui apakah hasil rancangan rangkaian yang sudah dibuat sudah sesuai dan memenuhi tujuan yang diinginkan atau belum. Proses ini biasanya dilakukan sebelum rancangan rangkaian diimplementasikan ke dalam FPGA. Jadi urutan sebagai berikut:

1. Pembuatan rancangan rangkaian (Metode schematic atau metode HDL).
2. Proses simulasi rancangan rangkaian.
3. Implementasi ke dalam FPGA.

Nantinya, proses simulasi berfungsi untuk mengetahui sekaligus menguji apakah rancangan

rangkaian yang telah dibuat mampu berjalan dengan baik atau tidak. Selain itu, lewat proses simulasi dapat diketahui bagaimana output dari rancangan rangkaian tadi. Selanjutnya, proses simulasi membutuhkan suatu bentuk stimulus/pemicu. Stimulus ini akan bertindak sebagai input awal bagi rancangan rangkaian yang hendak diuji. Kemudian, setelah diberikan stimulus maka rancangan rangkaian tersebut dapat diketahui Hal 1 dari 85 bagaimana hasil outputnya. Keseluruhan proses simulasi ini dilakukan dengan bantuan perangkat lunak (software) yang ada.

Pada umumnya, proses simulasi terbagi atas 2 bentuk yakni:

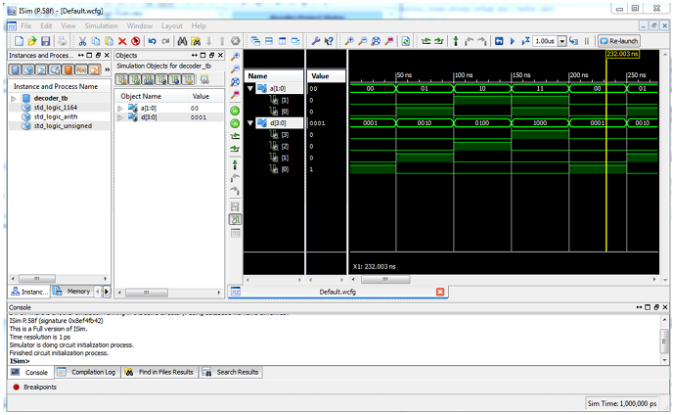
- Bentuk testbench.
- Bentuk timing diagram.

Timing Diagram

Proses simulasi dapat diketahui dengan membuat gambar timing diagram. Dengan menggunakan gambar timing diagram, dapat dilihat mengenai kondisi input, output serta hal-hal lain yang terkait. Bentuk timing diagram ini dibuat berdasarkan satuan waktu. Pada mulanya, untuk membuat timing diagram ini diperlukan stimulus untuk menentukan kondisi input awal. Selanjutnya, software simulator akan melakukan simulai guna menghasilkan kondisi output sesuai dengan kondisi input tadi.

Cara ini merupakan cara yang sederhana dan mudah dilakukan, apalagi bagi para pegguna awal FPGA, bila dibandingkan dengan bentuk testbench. Hal ini

disebabkan karena tampilan timing diagram yang berupa gambar sehingga memudahkan siapa saja untuk mengamati, menganalisa, dan menjelaskan proses simulasi.



Gambar 1. 1 Isim Simulator

Testbench

Selain cara pertama, dikenal pula bentuk simulasi yang lain yaitu testbench. Testbench adalah proses pengujian suatu rancangan rangkaian. Dalam proses perancangan rangkaian, testbench akan menguji design rangkaian apakah sudah sesuai atau belum. Testbench ini dilakukan dengan menggunaan file HDL (berbentuk kode, baik VHDL maupun Verilog). Sehingga berbeda dengan bentuk diagram yang menggunakan gambar sebagai tampilannya, testbench menggunakan kode-kode tulisan sebagai tampilannya. Berikut contoh testbench :

```
59
60      constant PERIOD : time := 10 ns;
61
62  BEGIN
63
64      -- Instantiate the Unit Under Test (UUT)
65      uut: circuit2 PORT MAP (
66            AT => AT,
67            BT => BT,
68            CT => CT,
69            DT => DT,
70            YT => YT
71          );
72
73      -- No clocks detected in port list. Replace <clock> below with
74      -- appropriate port name
75
76      -- Stimulus process
77      stim_proc: process
78      begin
79         wait for PERIOD;
80         AT<= '1';
81
82         wait for PERIOD;
83         BT<='1';
84
85         wait for PERIOD;
86         AT<='0';
87         CT<='1';
88
89         wait for PERIOD;
90         DT<='1';
91
92         wait for PERIOD;
93      end process;
94
95  END;
96
```

Gambar 1. 2 Test Bench

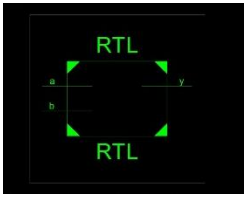
C. Percobaan

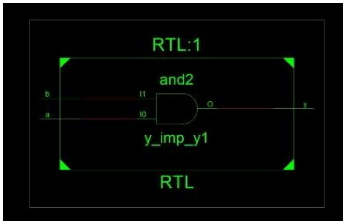
1. RTL dan Simulasi Test Bench (Rangkaian Kombinasional)

Program:

```
19  ----------------------------------------------
20  Library IEEE;
21  use IEEE.STD_LOGIC_1164.ALL;
22
23  entity RTL is
24     port(
25        a :in STD_LOGIC;
26        b :in STD_LOGIC;
27        y :out STD_LOGIC
28  );
29  end RTL;
30
31  architecture Behavioral of RTL is
32  begin
33     y <= a and b;
34  end Behavioral;
35
```

Hasil Percobaan:



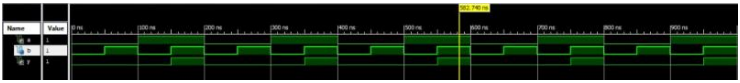


Program:

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY main_tb IS
END main_tb;

ARCHITECTURE behavior OF main_tb IS

    -- Component Declaration for the Unit Under Test (UUT)

    COMPONENT RTL
    PORT(
         a : IN  std_logic;
         b : IN  std_logic;
         y : OUT  std_logic
        );
    END COMPONENT;


   --Inputs
   signal a : std_logic := '0';
   signal b : std_logic := '0';

   --Outputs
   signal y : std_logic;
   -- No clocks detected in port list. Replace <clock> below with
   -- appropriate port name

--   constant <clock>_period : time := 10 ns;
```

```
BEGIN

   -- Instantiate the Unit Under Test (UUT)
   uut: RTL PORT MAP (
          a => a,
          b => b,
          y => y
        );

   -- Clock process definitions
--    <clock>_process :process
--    begin
--     <clock> <= '0';
--     wait for <clock>_period/2;
--     <clock> <= '1';
--     wait for <clock>_period/2;
--    end process;
--

-- Stimulus process
stim_proc:process
begin
-- hold reset state for 100 ns.
Wait for 100 ns;
a <=not a;
end process;

-- Stimulus process
stim_proc1:process
begin
-- hold reset state for 100 ns.
Wait for 50 ns;
b <=not b;
end process;

END;
```

Hasil Percobaan:

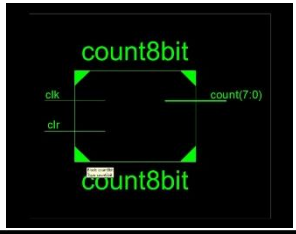


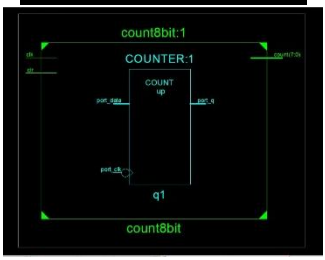
2. Rangkaian Sekuensial

Program:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_unsigned.ALL;

entity count8bit is
   Port ( clk : in STD_LOGIC;
          clr : in STD_LOGIC;
          count : out STD_LOGIC_VECTOR (7 downto 0));
end count8bit;

architecture Behavioral of count8bit is
   signal q : STD_LOGIC_VECTOR (7 downto 0);
begin
   process(clr, clk)
   begin
      if clr ='1' then q <= x"00";
      else
         if falling_edge(clk) then
            q <= q+1;
         end if;
      end if;
   end process;
   count <= q;
end Behavioral;
```

Hasil Percobaan:





Program:

```
--------------------------------------------------------------------------------
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY count_tb IS
END count_tb;

ARCHITECTURE behavior OF count_tb IS
   COMPONENT count8bit
   PORT(
      clk : IN std_logic;
      clr : IN std_logic;
      count : OUT std_logic_vector(7 downto 0)
      );
   END COMPONENT;
   --Inputs
   signal clk : std_logic := '0';
   signal clr : std_logic := '0';
   --Outputs
   signal count : std_logic_vector(7 downto 0);
   -- Clock period definitions
   constant clk_period : time := 10 ns;
BEGIN
   -- Instantiate the Unit Under Test (UUT)
   uut: count8bit PORT MAP (
      clk => clk,
      clr => clr,
      count => count
      );
   -- Clock process definitions
   clk_process :process
   begin
      clk <= '0';
      wait for clk_period/2;
      clk <= '1';
      wait for clk_period/2;
   end process;
```

```
   -- Stimulus process
   stim_proc: process
   begin
      clr <= '1'; wait for clk_period/2;
      clr <= '0'; wait for clk_period/2;
   wait for clk_period*10;
   -- insert stimulus here
   wait;
   end process;
END;
```

Hasil Percobaan:

D. Analisa

1. RTL dan Simulasi Test Bench (Rangkaian Kombinasional)

Pada praktikum ini, dilakukan praktik mengenai penggunaan rangkaian kombinasional dan sekuensial menggunakan ISIM Simulator. Langkah pertama dimulai dengan pembuatan program rangkaian logika AND dalam VHDL. Program ini dirancang dengan sebuah entity yang memiliki tiga port: dua input, yaitu a dan b yang bertipe data STD_LOGIC, serta satu output y dengan tipe data yang sama. Di dalam architecture yang diberi nama Behavioral, terdapat proses yang menerapkan fungsi logika AND antara input a dan b, kemudian hasilnya diberikan kepada output y menggunakan operator <=, yang berarti "assigned to". Dengan demikian, output y hanya bernilai 1 (atau 'H' dalam STD_LOGIC) apabila kedua input, a dan b, bernilai 1 (atau 'H'). Setelah program selesai dibuat, dilakukan pengujian rangkaian menggunakan testbench dengan mengatur periode waktu dari setiap input. Dari hasil pengujian ini,

diperoleh timing diagram yang menunjukkan kinerja rangkaian logika AND.

2. Rangkaian Sekuensial

Pada percobaan kedua, dilakukan pembuatan program untuk rangkaian sekuensial berupa counter 8-bit. Program ini menggambarkan sebuah counter yang bekerja berdasarkan sinyal clock (clk) dan sinyal reset (clr). Ketika sinyal reset aktif (clr = '1'), nilai counter akan dikembalikan ke 0. Namun, jika sinyal reset tidak aktif, counter akan bertambah satu setiap kali terjadi perubahan falling edge pada sinyal clock. Operasi increment ini dilakukan menggunakan operasi matematika pada tipe data STD_LOGIC_VECTOR, sehingga nilai counter bertambah satu pada setiap perubahan sinyal clock. Hasil dari counter ini disimpan dalam sinyal internal q yang bertipe STD_LOGIC_VECTOR 8-bit, kemudian diberikan ke output count. Setelah program dibuat, dilakukan pengujian fungsionalitasnya dengan testbench. Dalam testbench, terdapat proses clk_process yang menghasilkan sinyal clock (clk) dengan perubahan antara '0' dan '1' setiap setengah periode waktu yang telah ditentukan oleh clk_period. Sementara itu, proses stim_proc berfungsi memberikan stimulus pada desain yang diuji, seperti mengatur sinyal reset (clr) dan memberikan

penundaan sebelum simulasi berakhir. Dari hasil pengujian yang dilakukan, diperoleh timing diagram yang menggambarkan kinerja program counter 8-bit.

E. Kesimpulan

Pada percobaan pertama, yaitu implementasi rangkaian logika AND menggunakan VHDL, diperoleh kesimpulan bahwa simulasi dengan ISIM menunjukkan kebenaran operasi logika AND, di mana output hanya bernilai ‘1’ jika kedua input bernilai ‘1’. Dengan menggunakan pendekatan behavioral dalam arsitektur program, implementasi logika AND dapat dituliskan dengan lebih sederhana dan mudah dipahami. Pengujian menggunakan testbench juga membuktikan bahwa fungsi logika berjalan sesuai dengan ekspektasi, yang ditunjukkan melalui timing diagram yang dihasilkan.

Pada percobaan kedua, yaitu implementasi counter 8-bit berbasis rangkaian sekuensial, hasil simulasi menunjukkan bahwa counter bekerja sesuai dengan prinsip kerja sinyal clock dan reset. Saat sinyal reset aktif, nilai counter kembali ke nol, sedangkan ketika reset tidak aktif, counter meningkat satu unit setiap perubahan falling edge dari clock. Dengan menggunakan tipe data STD_LOGIC_VECTOR, operasi increment dapat dilakukan dengan mudah dalam lingkungan VHDL. Hasil simulasi testbench juga membuktikan bahwa sinyal clock

dan reset berfungsi sebagaimana mestinya, yang terlihat dari perubahan nilai counter dalam timing diagram. Dengan demikian, percobaan ini menunjukkan bahwa simulasi rangkaian sekuensial dalam VHDL dapat digunakan untuk menguji dan memverifikasi fungsionalitas sebelum implementasi dalam perangkat keras.